SurvDNN: Survival Deep Learning Models for Tabular Data

by Imad El Badisy

Abstract Deep learning is increasingly applied in survival analysis for high-dimensional data with complex effects. However, most implementations in R rely on Python backends, limiting accessibility and integration with R workflows. We present survdnn, a fully native R package for deep neural survival modeling, built on the torch library. The package offers a formula interface consistent with R conventions, supports multiple loss functions (Cox, Cox with L2 penalty, Accelerated Failure Time, Cox-Time), and provides utilities for prediction, model evaluation, cross-validation, and hyperparameter tuning. Benchmarks on real data show that survdnn performs competitively against classical models such as Cox regression and Random Survival Forests. This package extends the scope of survival analysis to deep learning within the R ecosystem.

1 Introduction

1.1 Background and motivation

Survival analysis is a foundational methodology in biomedical and clinical research, constituting the methodological groundwork for applications such as oncology prognosis, cardiovascular risk stratification, and reliability engineering (Wiegrebe et al., 2024). Classical statistical models, most notably the Cox proportional hazards (PH) model, have long been favored for their interpretability and solid theoretical foundation (Cox, 1972). However, these models impose restrictive assumptions, including linear covariate effects and PH, which are frequently violated in real-world applications (Heagerty and Zheng, 2005; Batson et al., 2016).

To address these limitations, machine learning algorithms have been increasingly adopted as flexible modeling approaches that relax restrictive assumptions and often yield improved predictive accuracy. Among these methods, deep learning has emerged as a particularly powerful paradigm, capable of capturing complex non-linear relationships and high-dimensional feature interactions that are difficult to capture with classical approaches (Wiegrebe et al., 2024; Bengio et al., 2021).

1.2 Advances in deep survival models

The evolution of deep survival models began with early efforts such as that of Faraggi and Simon (Faraggi and Simon, 1995), who replaced the linear component of Cox regression with a shallow neural network. However, these early models failed to consistently outperform classical models in terms of discrimination (Xiang et al., 2000; Sargent, 2001). A significant advancement came with `DeepSurv` (Katzman et al., 2018), which introduced modern neural optimization techniques to improve predictive accuracy under the PH setting.

Recent deep learning methods have achieved state-of-the-art performance in survival analysis, particularly through models such as `DeepSurv`, `DeepHit` (Lee et al., 2018), and `Cox-Time` (Kvamme et al., 2019), which have been successfully applied to real-world clinical datasets. These methods are summarized in recent reviews (Wiegrebe et al., 2024).

To model non-PH and flexible survival distributions, `DeepHit` (Lee et al., 2018) introduced a discrete-time neural approach with time-varying, non-linear effects. The `pycox` library unified `DeepSurv`, `DeepHit`, and `Cox-Time` using `PyTorch` (Paszke et al., 2019), while `TorchSurv` (Monod et al., 2024) more recently offered a lower-level API for neural Cox and AFT models with customizable losses.

1.3 Statement of need

Despite the growing adoption of deep learning methods for survival analysis, the R ecosystem lacks native tools for training and deploying deep survival models. Existing solutions, such as the **survivalmodels** package (Sonabend, 2024), rely on Python backends accessed via **reticulate** (Ushey et al., 2025), which introduces installation complexity and versioning issues. This dependency has limited the accessibility of deep survival modeling for biostatisticians and clinical researchers who rely primarily on R.

1.4 Contribution

To address this gap, we developed **survdnn**, a fully native R package for deep neural network-based survival analysis. Built on the **torch** backend (Falbel and Luraschi, 2025), **survdnn** provides a high-level formula interface, supports multiple loss functions including Cox, Cox-L2, AFT, and Cox-Time, and integrates seamlessly with modern R workflows. The package includes training and prediction utilities, built-in evaluation metrics such as the concordance index (C-index), time-dependent Brier score, and Integrated Brier Score (IBS), as well as functions for cross-validation, hyperparameter tuning, and regularization mechanisms including dropout, batch normalization, and early-stopping callbacks. **survdnn** complements the rich foundational survival modeling tools in R by extending them into the deep learning era without leaving the R ecosystem.

2 Implementation

2.1 Package overview

At its core, **survdnn** is built around the `survdnn()` function, which fits a deep neural network for right-censored survival data using R's formula interface. Users specify the network architecture through an integer vector of hidden layer sizes and select the activation function from a set of supported options, including `"relu"`, `"leaky_relu"`, `"tanh"`, `"sigmoid"`, `"gelu"`, `"elu"`, and `"softplus"`. The training objective is defined via the `loss` argument, with support for `"cox"`, `"cox_l2"`, `"aft"`, and `"coxtime"`, while core hyperparameters such as the learning rate and number of epochs remain fully user-configurable. The training pipeline further incorporates regularization and optimization mechanisms such as dropout, batch normalization, and user-defined callbacks (including early stopping based on training loss dynamics) to improve stability and generalization.

Model predictions are computed using `predict.survdnn()`, which supports three output types: the raw linear predictor (`"lp"`), survival probabilities at specified time points (`"survival"`), and cumulative risk at a given time (`"risk"`). Predictions can be returned either as numeric vectors or as data frames with one column per evaluation time.

The package includes built-in utilities for evaluation and resampling. The `evaluate_survdnn()` function computes common and built-in performance metrics. The `cv_survdnn()` function performs stratified k-fold cross-validation to preserve event censoring balance across folds, with results summarized using `summarize_cv_survdnn()`.

Hyperparameter tuning is supported through two complementary functions. The `tune_survdnn()` function performs k-fold cross-validation over a user-defined hyperparameter grid and selects the best configuration according to a chosen evaluation metric (the first metric is used for selection), optionally refitting the final model on the full dataset. In contrast, `gridsearch_survdnn()` evaluates the same type of hyperparameter grid under a fixed training-validation split: each candidate model is fitted on the training set and assessed on a user-supplied validation set, without internal resampling or automatic refitting.

Missing data are handled explicitly through a user-controlled policy (`na_action = "omit"` or `"fail"`), with informative reporting of excluded observations when applicable.

Finally, `plot.survdnn()` provides a visualization method for predicted survival curves using `ggplot2` style (Wickham, 2016), with options to group curves by a categorical variable and to display either individual or averaged curves.

2.2 Model architecture

The model architecture in **survdnn** is defined as a sequence of k transformation layers applied to the input features $\mathbf{x}$:

$$f_\theta(\mathbf{x}) = L_k \circ \phi_{k-1} \circ \cdots \circ \phi_1 \circ L_1(\mathbf{x})$$

Each stage consists of a linear transformation followed by non-linear activation and regularization components. The linear transformation at layer k is expressed as:

$$L_k(\cdot) = W_k \mathbf{h}^{(k-1)} + b_k$$

where W_k is the weight matrix and b_k the bias vector. The transformed output is then passed through a composition of non-linear operations. Specifically, each layer includes an activation function ϕ, batch normalization to stabilize learning, and dropout to prevent overfitting. Together, a hidden layer is computed as:

$$\mathbf{h}^{(k)} = \text{Dropout}\left(\phi\left(\text{BatchNorm}\left(W_k \mathbf{h}^{(k-1)} + b_k\right)\right)\right)$$

The final layer outputs a scalar prediction $f_\theta(\mathbf{x})$, interpreted based on the selected loss function: as log-risk under Cox-based losses, log-event time under the AFT loss, or time-dependent log-risk under the Cox-Time loss.

Before training, a model matrix is created from the user-specified formula interface. This step expands factor variables into dummy indicators and extracts the design matrix.

2.3 Loss functions

In survival machine learning, the loss function defines the model's training objective and dictates how it handles censoring, time-to-event outcomes, and individual risk dynamics (Wiegrebe et al., 2024). This choice critically influences convergence, calibration, and generalization (Katzman et al., 2018).

The **survdnn** package implements four core loss functions tailored for right-censored survival data, covering both PH and non-PH settings:

Cox Partial Likelihood Loss ("cox"): The standard Cox loss implements the partial likelihood from the Cox PH model (Cox, 1972). It is the basis of classical methods and modern neural-based implementations like `DeepSurv` (Katzman et al., 2018) and `pycox` (Kvamme et al., 2019).

The model assumes a multiplicative hazard function:

$$\lambda(t \mid \mathbf{x}) = \lambda_0(t) \cdot \exp(f_\theta(\mathbf{x})) = \lambda_0(t) \cdot e^{\eta}$$

where $f_\theta(\mathbf{x}) = \eta$ is a learned log-risk score and $\lambda_0(t)$ is an unspecified baseline hazard. To avoid estimating $\lambda_0(t)$, Cox proposed the partial likelihood:

$$\mathcal{L}_{\text{cox}}(\theta) = \prod_{i:\delta_i=1} \frac{e^{\eta_i}}{\sum_{j \in \mathcal{R}(t_i)} e^{\eta_j}}$$

The negative log of this expression is minimized during training. In **survdnn**, this is implemented using `torch_logcumsumexp()` to ensure numerical stability and efficient batch-wise computation.

The Cox loss learns relative risks, not calibrated survival probabilities. However, post-processing via Breslow estimation is performed to obtain full survival curve estimates (Breslow, 1972).

L2-Regularized Cox Loss (`"cox_l2"`): This variant augments the Cox loss with an L2 penalty applied directly to the predicted log-risk scores:

$$\mathcal{L}_{\text{cox-l2}}(\theta) = \mathcal{L}_{\text{cox}}(\theta) + \lambda \cdot \frac{1}{n} \sum_{i=1}^{n} \eta_i^2$$

This ridge-type regularization stabilizes optimization and reduces overfitting, particularly in small-sample or high-dimensional settings (Friedman et al., 2010).

Unlike classical weight decay, the penalty acts on model outputs rather than internal parameters, preserving architectural flexibility while directly constraining extreme risk predictions.

Accelerated Failure Time Loss (`"aft"`): The AFT option in **survdnn** implements a log-normal AFT model with right-censoring, trained by minimizing the censored negative log-likelihood:

$$\log(T_i) = \mu_i + \sigma Z_i, \qquad Z_i \sim \mathcal{N}(0,1)$$

where $\mu_i = f_\theta(\mathbf{x}_i) + \text{aft\_loc}$ is a subject-specific location parameter predicted by the neural network, $\sigma > 0$ is a global scale parameter, and `aft_loc` is an optional centering constant used to improve numerical stability.

For an observed time t_i and event indicator δ_i, the contribution to the negative log-likelihood is:

$$\ell_i(\theta) = \begin{cases} \log t_i + \log \sigma + \frac{1}{2} z_i^2 & \delta_i = 1 \\ -\log S(z_i) & \delta_i = 0 \end{cases}$$

where

$$z_i = \frac{\log t_i - \text{aft\_loc} - f_\theta(\mathbf{x}_i)}{\sigma}, \quad S(z) = 1 - \Phi(z)$$

and $\Phi(\cdot)$ denotes the standard normal cumulative distribution function.

The overall loss minimized during training is the average censored negative log-likelihood:

$$\mathcal{L}_{\text{aft}}(\theta) = \frac{1}{n} \sum_{i=1}^{n} \ell_i(\theta)$$

In **survdnn**, the scale parameter σ can either be fixed or learned jointly with network parameters through a differentiable reparameterization ($\sigma = \exp(\log \sigma)$). This formulation enables the AFT model to fully use both event and censored observations while preserving the direct time-scale interpretation of AFT models.

Cox-Time Loss (`"coxtime"`): The Cox-Time model extends the Cox framework by allowing covariate effects to vary with time (Kvamme et al., 2019). The log-relative hazard is modeled as:

$$g_\theta(t, \mathbf{x}) = \log\left(\frac{h(t \mid \mathbf{x})}{h_0(t)}\right)$$

yielding the hazard:

$$h(t \mid \mathbf{x}) = h_0(t) \exp(g_\theta(t, \mathbf{x}))$$

The corresponding loss is the time-dependent Cox partial likelihood:

$$\mathcal{L}_{\text{coxtime}}(\theta) = -\sum_{i:\,\delta_i=1}\left[g_\theta(t_i,\mathbf{x}_i) - \log\sum_{j\in\mathcal{R}(t_i)}\exp\big(g_\theta(t_i,\mathbf{x}_j)\big)\right]$$

In **survdnn**, this loss is implemented using batched evaluation over event times and risk sets. While Cox-Time offers the greatest modeling flexibility among the available losses, it is also the most computationally demanding due to repeated evaluation of risk sets across time.

2.4 Prediction interface

Once a **survdnn** model has been trained, predictions can be obtained using the generic `predict()` method. This function supports multiple output types to serve different goals in survival analysis:

- *Linear predictors* (`type = "lp"`): returns the log-risk score $\eta = f_\theta(\mathbf{x})$, used for risk ranking and concordance index evaluation.
- *Survival probabilities* (`type = "survival"`): returns $\hat{S}(t \mid \mathbf{x})$, the estimated survival function evaluated at user-specified time points.
- *Cumulative risk* (`type = "risk"`): returns the cumulative incidence $\hat{F}(t) = 1 - \hat{S}(t)$, the probability of experiencing the event by time t.

Predictions are returned either as numeric vectors for single time points or as data frames with one column per evaluation time.

The underlying computation depends on the loss function used during training. For models trained with `"cox"` and `"cox_l2"`, the baseline cumulative hazard $\hat{H}_0(t)$ is estimated using the Breslow estimator.

Survival probabilities are then computed as:

$$\hat{S}(t \mid \mathbf{x}) = \exp\big(-\hat{H}_0(t)\exp(\eta)\big)$$

For `"aft"`, predictions are based on the fitted log-normal accelerated failure time model, using the estimated scale parameter $\hat{\sigma}$:

$$\hat{S}(t \mid \mathbf{x}) = 1 - \Phi\left(\frac{\log t - f_\theta(\mathbf{x})}{\hat{\sigma}}\right)$$

where Φ denotes the standard normal cumulative distribution function.

For `"coxtime"`, predictions follow the approach proposed by (Kvamme et al., 2019), using a time-dependent neural log-risk function $g(t,\mathbf{x})$. Survival probabilities are obtained by accumulating predicted hazards over a discrete time grid:

$$\hat{S}(t \mid \mathbf{x}) = \prod_{s\le t}\exp\big(-\hat{\lambda}(s \mid \mathbf{x})\big)$$

Survival prediction under Cox-Time requires numerical integration of the time-dependent hazard over a user-defined time grid. While training relies only on observed event times, prediction involves evaluating the neural risk function across grid points for each individual, which can become computationally expensive for large datasets or fine time grids.

2.5 Training workflow

The `survdnn()` function implements a complete training workflow for deep neural networks on right-censored survival data. It relies on R's formula interface to construct design matrices, applies Z-score standardization for numerical stability, and converts inputs into `torch_tensor()` objects for efficient automatic differentiation using the **torch** backend (Keydana, 2023).

Model architectures are defined via `build_dnn()`, which constructs a fully connected multilayer perceptron with user-configurable depth and width. Each hidden layer may optionally include batch normalization and dropout, enabling regularization and improved optimization stability. Non-linear activations are applied uniformly across layers and selected from a predefined set of commonly used functions.

The training objective is specified via the `loss` argument (`"cox"`, `"cox_l2"`, `"aft"`, or `"coxtime"`), each implemented as a differentiable loss function compatible with backpropagation. Optimization is performed using full-batch gradient descent with a choice of optimizers, including Adam, AdamW, SGD, RMSprop, and Adagrad, all exposed through a unified interface.

The empirical risk minimized during training is given by:

$$\mathcal{L}(\theta) = \frac{1}{n} \sum_{i=1}^{n} \ell(f_\theta(\tilde{\mathbf{x}}_i), t_i, \delta_i)$$

where $\ell(\cdot)$ denotes the selected survival loss and $\tilde{\mathbf{x}}_i$ are standardized covariates.

To improve training robustness and prevent overfitting, **survdnn** supports user-defined callback functions, including early stopping based on the training loss trajectory. Callbacks are evaluated at each epoch and may interrupt training once predefined criteria are met. Reproducibility is ensured by a unified `.seed` argument that synchronizes R and **torch** random number generators.

The fitted model is returned as an S3 object of class `"survdnn"`, containing the trained neural network, preprocessing metadata, optimizer configuration, and the full loss history. This object integrates seamlessly with the `predict()` method to compute predictions.

2.6 Evaluation metrics

survdnn supports standard survival metrics for assessing both discrimination and calibration at user-defined evaluation time points. Discrimination is evaluated using the C-index (Harrell et al., 1982), which measures the model's ability to correctly rank individuals by risk and is maximized during evaluation and hyperparameter tuning.

Calibration is assessed using the time-dependent Brier score (Graf et al., 1999) and the IBS, both computed with inverse probability of censoring weighting to account for right-censoring (Zhou et al., 2023). These metrics quantify the accuracy of predicted survival probabilities and are minimized during evaluation and hyperparameter tuning.

2.7 Cross-validation

Cross-validation is performed using the `cv_survdnn()` function, which implements stratified k-fold partitioning based on the event indicator to preserve the proportion of observed events and censored observations across folds. Time-to-event values are not stratified, in accordance with standard practice in survival resampling.

Within each fold, models are trained using `survdnn()` on the training split and evaluated on the held-out data using `evaluate_survdnn()` at user-specified time points and metrics. Results are returned in a tidy tabular format, enabling transparent benchmarking, hyperparameter selection, and performance comparison while reducing optimistic bias (Varma and Simon, 2006).

2.8 Tuning

Hyperparameter tuning is crucial in deep survival modeling, where censoring can destabilize optimization and introduce noisy gradients, and small datasets with few events further increase overfitting risk (Gensheimer and Narasimhan, 2019).

survdnn provides `tune_survdnn()` to perform grid-based k-fold cross-validation across key hyperparameters, including:

- **Architecture**: hidden layer sizes (`hidden`)
- **Activation**: `"relu"`, `"leaky_relu"`, `"tanh"`, `"sigmoid"`, `"gelu"`, `"elu"`, `"softplus"`
- **Optimization**: learning rate (`lr`), number of epochs (`epochs`)
- **Loss function**: `"cox"`, `"cox_l2"`, `"aft"`, `"coxtime"`

This design follows best practices in regularized learning (Friedman et al., 2010) and deep neural optimization (Bengio, 2012). An optional `refit = TRUE` automatically retrains the best configuration and fits the model on the full dataset in an AutoML style.

2.9 Sensitivity to censoring

Because censoring directly affects the effective sample size and the information available for risk estimation, we investigated the sensitivity of **survdnn** models to increasing levels of right censoring. Using a controlled simulation design, we evaluated predictive discrimination (C-index) and calibration (IBS) across censoring rates ranging from 10% to 90%.

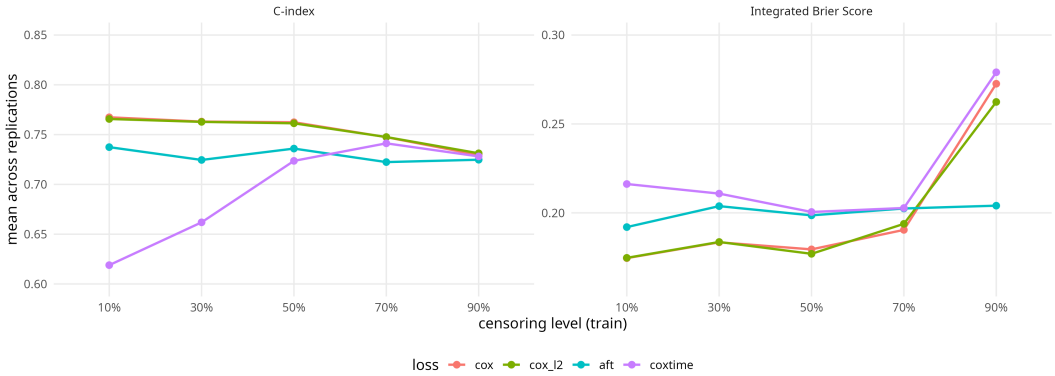


Figure 1: Censoring sensitivity analysis. C-index and IBS of survdn models across increasing right-censoring rates (10-90%) for the four implemented loss functions.

Across losses, discrimination (C-index) decreases only moderately with increasing censoring, whereas calibration (IBS) is most affected at extreme censoring (Figure 1). Cox-based losses (`"cox"` and `"cox_l2"`) show stable C-index up to $\approx$ 50% censoring, followed by a gradual decline, while IBS remains low up to $\approx$ 70% but increases sharply at 90% censoring. The AFT loss exhibits comparatively stable performance across censoring levels for both C-index and IBS, and yields the lowest IBS under very high censoring (90%). In contrast, the Cox-Time loss shows poor discrimination at low censoring but improves markedly up to $\approx$ 70%, while maintaining higher IBS than the other losses and deteriorating strongly at 90% censoring, consistent with increased variance under limited effective sample size.

These results suggest that, in practice, Cox-based losses are preferable under low to moderate censoring, offering a favorable balance between discrimination and calibration. The AFT loss demonstrates robust calibration across censoring levels, including under very high censoring, but yields comparatively lower discrimination, indicating that its use should be guided by the relative importance of ranking versus probability accuracy. Highly

flexible models such as Cox-Time require larger effective sample sizes and careful calibration assessment, as their apparent gains in discrimination at moderate censoring are offset by substantially poorer calibration under extreme censoring.

2.10 Computation backend

Since **survdnn** is built on the **torch** backend, it supports both CPU and GPU execution. The computation device can be selected via `.device = c("auto", "cpu", "cuda")`, where `"auto"` uses CUDA when available and otherwise falls back to CPU. On CPU, performance benefits from multithreaded linear algebra provided by the underlying BLAS/LAPACK implementation.

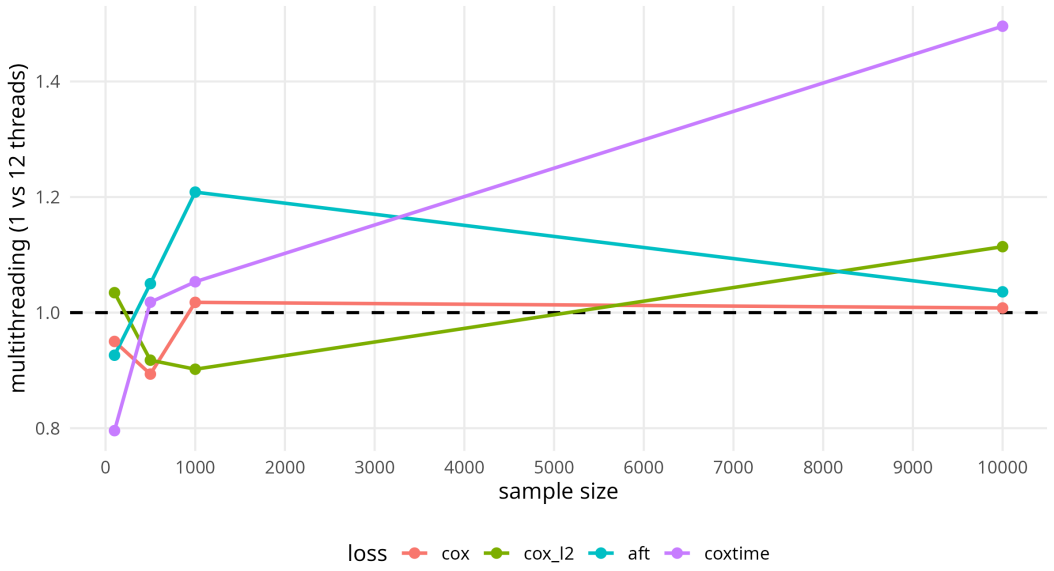


Figure 2: CPU multithreading speedup across loss functions. Relative training-time speedup comparing multi-threaded and single-thread CPU execution as a function of sample size for Cox, Cox-L2, AFT, and Cox-Time losses.

To characterize computational behavior, we evaluated training time across increasing sample sizes and compared single-thread and multi-thread CPU execution for different loss functions (Figure 2). Multithreading benefits vary markedly across losses and depend strongly on sample size. Cox-based losses exhibit little to no speedup, with performance remaining close to single-thread execution even at large sample sizes, reflecting the dominance of global, sequential risk-set operations. The AFT loss shows moderate speedup already at small to moderate sample sizes, but scaling gains diminish as sample size increases, consistent with dense likelihood computations that limit parallel efficiency. In contrast, the Cox-Time loss demonstrates substantial and increasing speedup with sample size, achieving the largest gains under multithreading, indicating favorable parallelization of its time-dependent risk evaluations despite higher overhead at small sample sizes.

2.11 Reproducibility

Reproducibility in **survdnn** requires controlling both R-level and **torch**-level sources of randomness. While `set.seed()` controls randomness in R, neural network training relies on **torch**, which maintains its own random number generator for operations such as weight initialization and dropout.

To simplify reproducible workflows, **survdnn** exposes a `.seed` argument in its main fitting and resampling functions (including `survdnn()`, `cv_survdnn()`, and `tune_survdnn()`). When provided, this argument internally synchronizes both R and **torch** random number generators to ensure fully reproducible model training and evaluation.

3 Usage examples

We illustrate the use of **survdnn** through two case studies: a complete modeling pipeline on the veteran's lung cancer dataset and a benchmark on the real-world METABRIC breast cancer dataset.

3.1 Example 1: API demonstration

This example uses the Veteran's lung cancer dataset from the **survival** package (Terry M. Therneau and Patricia M. Grambsch, 2000) to illustrate a typical **survdnn** workflow: fit a model, inspect the training configuration, compute survival predictions at user-defined time points, evaluate with standard metrics (C-index, IBS, Brier), perform cross-validation, tune hyperparameters, and visualize training dynamics and survival curves.

First, we fit a Cox-loss model with a modest multilayer architecture:

```
pacman::p_load(survdnn, survival, dplyr, ggplot2, tibble, randomForestSRC, partykit, party, pec, rsam

veteran <- survival::veteran

mod <- survdnn(
  Surv(time, status) ~ age + karno + celltype,
  data = veteran,
  hidden = c(32, 16),
  epochs = 300,
  loss = "cox",
  verbose = TRUE,
  .seed  = 123
  )

#> Epoch 50 - Loss: 3.888980
#>
#> Epoch 100 - Loss: 3.856431
#>
#> Epoch 150 - Loss: 3.822445
#>
#> Epoch 200 - Loss: 3.839233
#>
#> Epoch 250 - Loss: 3.779270
#>
#> Epoch 300 - Loss: 3.807069
```

We can summarise the model to inspect the architecture, preprocessing, and training setup:

```
summary(mod)

#>
#> Formula:
#>   Surv(time, status) ~ age + karno + celltype
#> <environment: 0x109680e90>
#>
#> Model architecture:
#>   Hidden layers:  32 : 16
#>   Activation:  relu
#>   Dropout:  0.3
#>   Batch norm:  TRUE
```

```
#>   Final loss:  3.807069
#>
#> Training summary:
#>   Epochs:  300
#>   Learning rate:  1e-04
#>   Loss function:  cox
#>   Optimizer:  adam
#>   Device:  cpu
#>   NA action:  omit
#>
#> Data summary:
#>   Observations (used/total): 137 / 137
#>  Predictors (5): age, karno, celltypesmallcell, celltypeadeno, celltypelarge
#>   Time range: [ 1, 999 ]
#>   Events / censored:  128  /  9
#>   Event rate:  93.4%
#>   Predictors standardized:  yes
```

Next, we can obtain survival probabilities at clinically meaningful horizons (30, 90, 180 days):

```
times_eval <- c(30, 90, 180)
pred_surv <- predict(mod, newdata = veteran, type = "survival", times = times_eval)
head(pred_surv)
```

```
#>        t=30      t=90     t=180
#> 1 0.8500299 0.6484395 0.3320248
#> 2 0.8908178 0.7347433 0.4563401
#> 3 0.8837166 0.7192316 0.4322179
#> 4 0.8764463 0.7035644 0.4086562
#> 5 0.8869652 0.7263022 0.4431147
#> 6 0.7840707 0.5228148 0.1919285
```

Model performance can be assessed via the C-index and the IBS:

```
eval_res <- evaluate_survdnn(mod, metrics = c("cindex", "ibs"), times = times_eval)
print(eval_res)
```

```
#> # A tibble: 2 x 2
#>   metric value
#>   <chr>  <dbl>
#> 1 cindex 0.745
#> 2 ibs    0.169
```

The Brier score can also be computed at specific time points (or over a grid, if desired):

```
times_eval <- seq(min(veteran$time), max(veteran$time), 10)
eval_res <- evaluate_survdnn(mod, metrics = "brier", times = times_eval)
print(eval_res)
```

```
#> # A tibble: 100 x 3
#>    metric  time    value
#>    <chr>  <dbl>    <dbl>
#>  1 brier      1 0.000073
#>  2 brier     11 0.0880
#>  3 brier     21 0.150
```

```
#>  4 brier     31 0.181
#>  5 brier     41 0.192
#>  6 brier     51 0.194
#>  7 brier     61 0.199
#>  8 brier     71 0.198
#>  9 brier     81 0.193
#> 10 brier     91 0.188
#> # i 90 more rows
```

To assess stability, we can run k-fold cross-validation with predefined metrics and time points:

```
cv_results <- cv_survdnn(
  Surv(time, status) ~ age + karno + celltype, data = veteran,
  times = c(30, 90, 180),
  metrics = c("ibs"),
  folds = 3,
  hidden = c(16, 8),
  loss = "cox",
  epochs = 100,
  .seed  = 123
  )

#> Epoch 50 - Loss: 3.587685
#>
#> Epoch 100 - Loss: 3.621760
#>
#> Epoch 50 - Loss: 3.680103
#>
#> Epoch 100 - Loss: 3.627799
#>
#> Epoch 50 - Loss: 3.595493
#>
#> Epoch 100 - Loss: 3.660711

print(cv_results)

#> # A tibble: 3 x 3
#>    fold metric value
#>   <int> <chr>  <dbl>
#> 1     1 ibs    0.230
#> 2     2 ibs    0.226
#> 3     3 ibs    0.230
```

Hyperparameter tuning can be automated over a small grid. Below we vary architecture, optimiser settings, and loss:

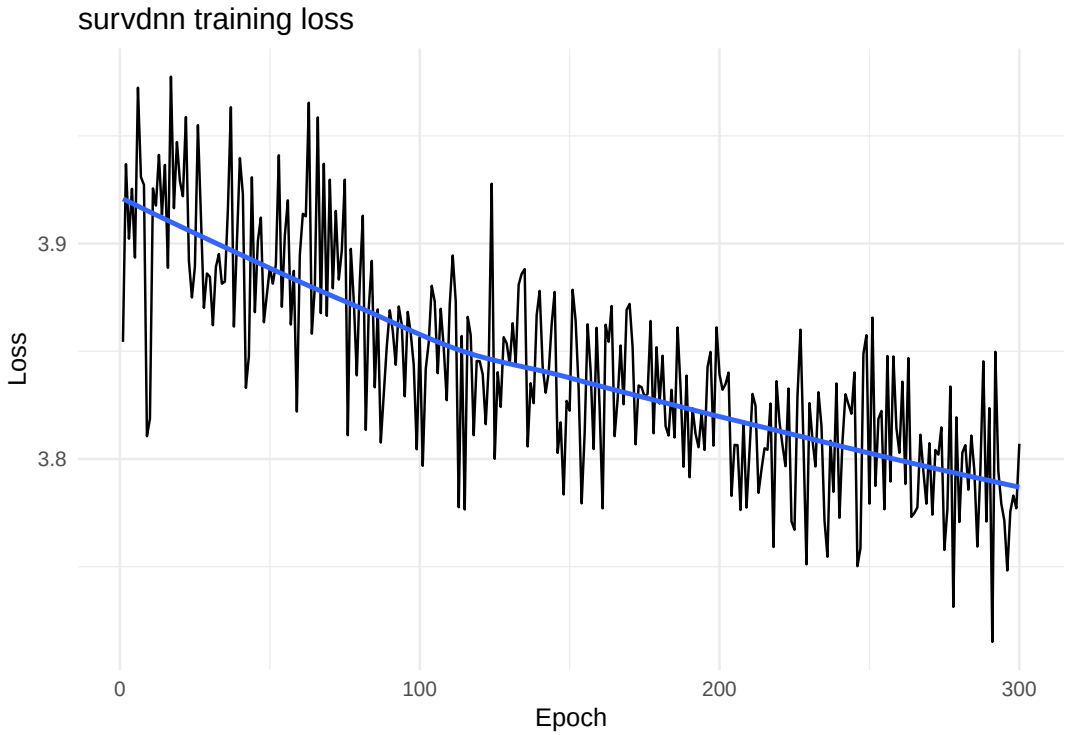
```
grid <- list(
  hidden     = list(c(16, 32, 64, 16)),
  lr         = c(1e-3),
  activation = c("relu", "softplus"),
  epochs     = c(100),
  loss       = c("cox_l2", "aft")
)

tune_res <- tune_survdnn(
```

```
  Surv(time, status) ~ age + karno + celltype,
  data = veteran,
  times = c(30, 90, 180),
  metrics = "cindex",
  param_grid = grid,
  folds = 3,
  refit = FALSE,
  return = "all"
  )

#> Epoch 50 - Loss: 4.482142
#>
#> Epoch 100 - Loss: 4.390209
#>
#> Epoch 50 - Loss: 4.638953
#>
#> Epoch 100 - Loss: 4.503160
#>
#> Epoch 50 - Loss: 4.460026
#>
#> Epoch 100 - Loss: 4.299024
#>
#> Epoch 50 - Loss: 3.427203
#>
#> Epoch 100 - Loss: 3.418092
#>
#> Epoch 50 - Loss: 3.485920
#>
#> Epoch 100 - Loss: 3.404513
#>
#> Epoch 50 - Loss: 3.368922
#>
#> Epoch 100 - Loss: 3.319427
#>
#> Epoch 50 - Loss: 4.524547
#>
#> Epoch 100 - Loss: 4.475084
#>
#> Epoch 50 - Loss: 4.673272
#>
#> Epoch 100 - Loss: 4.551622
#>
#> Epoch 50 - Loss: 4.516170
#>
#> Epoch 100 - Loss: 4.417117
#>
#> Epoch 50 - Loss: 3.504852
#>
#> Epoch 100 - Loss: 3.519552
#>
#> Epoch 50 - Loss: 3.490904
#>
#> Epoch 100 - Loss: 3.530700
#>
#> Epoch 50 - Loss: 3.468363
#>
```

```
#> Epoch 100 - Loss: 3.391495

benchmark_results <- summarize_tune_survdnn(tune_res, by_time = TRUE)

tuning_results_clean <- benchmark_results %>%
  mutate(
    hidden = sapply(hidden, function(x) paste(x, collapse = "-")),
    across(where(is.numeric), ~ round(.x, 3))
  )

tuning_results_clean

#> # A tibble: 4 x 8
#>   hidden         lr activation epochs loss   metric  mean    sd
#>   <chr>       <dbl> <chr>       <dbl> <chr>  <chr>  <dbl> <dbl>
#> 1 16-32-64-16 0.001 softplus      100 aft    cindex 0.737 0.072
#> 2 16-32-64-16 0.001 softplus      100 cox_l2 cindex 0.736 0.074
#> 3 16-32-64-16 0.001 relu          100 aft    cindex 0.723 0.079
#> 4 16-32-64-16 0.001 relu          100 cox_l2 cindex 0.715 0.06
```

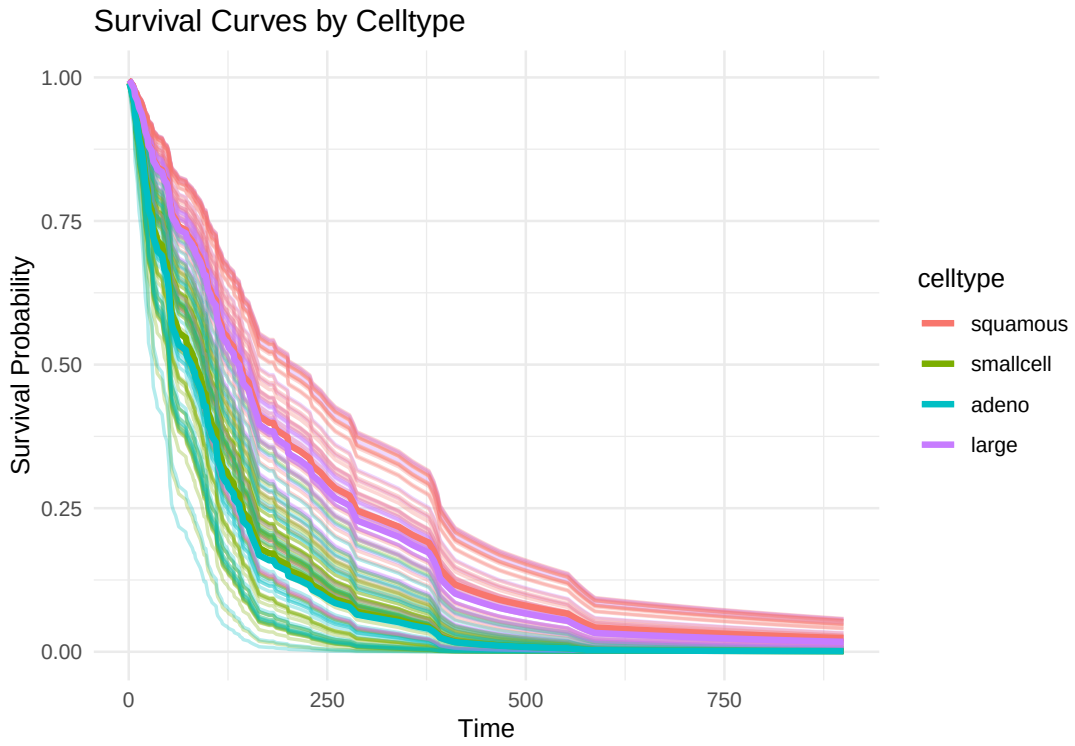
Training dynamics can be visualised using the built-in `plot_loss()` helper:

```
plot_loss(mod, smooth = TRUE)
```

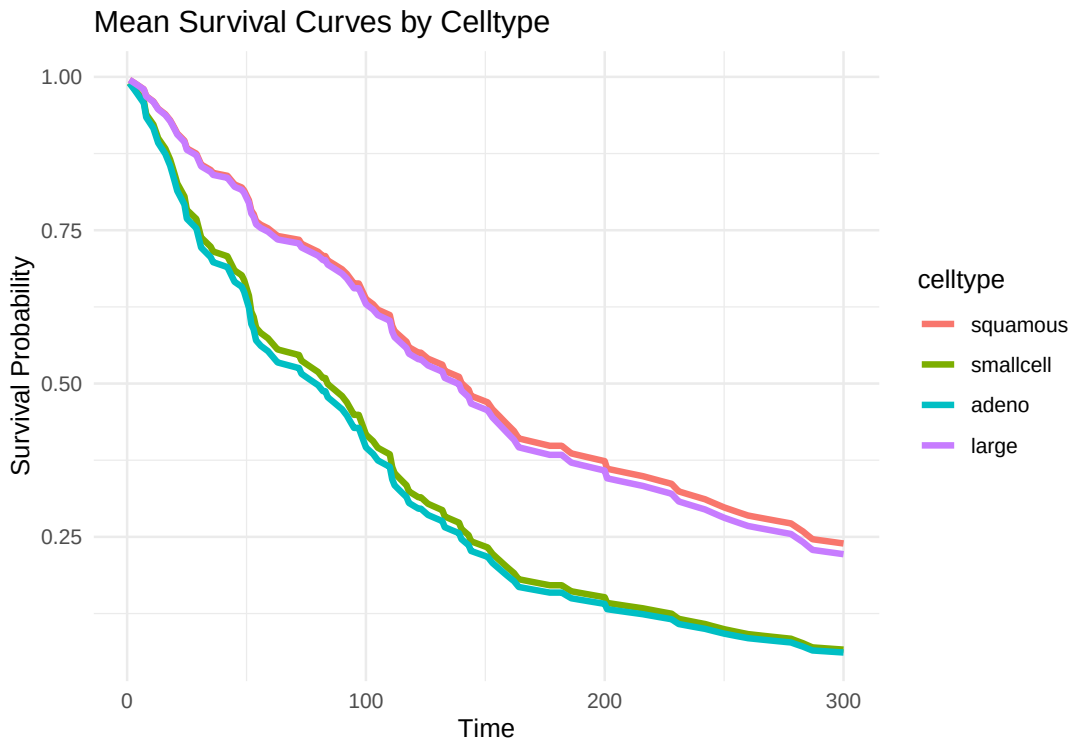


Finally, we visualise survival curves by cancer cell type, showing both individual and mean summaries:

```
plot(mod, group_by = "celltype", times = 1:900) +
  ggtitle("Survival Curves by Celltype")
```




```
plot(mod, group_by = "celltype", times = 1:300, plot_mean_only = TRUE) +
  ggtitle("Mean Survival Curves by Celltype")
```



3.2 Example 2: METABRIC benchmark

Here we benchmark **survdnn** against Cox PH, Random Survival Forests (RSF), and Conditional Inference Forests (CForest) using five-fold cross-validation on the METABRIC breast cancer dataset (Mukherjee et al., 2018). All models are trained on identical splits and evaluated with the C-index at predefined time points.

```
#> # A tibble: 4 x 3
```

```
#>   model   mean_cindex sd_cindex
#>   <chr>         <dbl>     <dbl>
#> 1 SurvDNN       0.611     0.029
#> 2 CoxPH         0.604     0.029
#> 3 RSF           0.554     0.033
#> 4 CForest       0.52      0.044
```

4 Discussion

The **survdnn** package fills an important gap in the R ecosystem by providing a fully native, **torch**-based framework for deep survival modeling (Falbel and Luraschi, 2025), without relying on Python backends. This design choice improves installation reliability and facilitates seamless integration with established R workflows for statistical modeling and reproducible research.

Across multiple benchmarks and sensitivity analyses, **survdnn** demonstrates competitive predictive performance relative to established survival models, including Cox PH models (Cox, 1972), RSF (Ishwaran et al., 2008), and CForest (Hothorn et al., 2006). The availability of multiple likelihood-based loss functions enables analysts to adapt modeling assumptions to PH, non-PH, non-linear covariate effects, and time-varying effects within a unified deep learning framework. All implemented losses explicitly account for right-censoring through either partial likelihoods (Cox, L2-penalised Cox, Cox-Time) or fully specified censored likelihoods (AFT).

Additional investigations highlight practical trade-offs in both statistical robustness and computational behavior. Sensitivity analyses across increasing censoring levels (Figure 1) show that discrimination and calibration degrade as expected with reduced effective sample size, with differences across loss functions. Complementary runtime experiments (Figure 2) demonstrate that computational behavior varies substantially by loss function: Cox-based losses exhibit low computational overhead but derive little benefit from multithreaded execution, Cox-Time incurs higher per-iteration cost due to repeated evaluation of time-dependent risk scores yet scales efficiently with parallel execution at larger sample sizes, and AFT likelihoods involve dense computations that benefit from parallelism at moderate sizes but show limited scalability as sample size increases.

The current scope of **survdnn** is intentionally focused on tabular data, using multilayer perceptron architectures (Bengio et al., 2021) that are well aligned with the structure of most survival analysis applications. This design choice prioritizes stability and compatibility with classical survival modeling workflows. Training is performed using full-batch optimization, which favors reproducibility and stable convergence but may limit scalability to very large datasets.

Ongoing work maps **survdnn** to the `mlr3` ecosystem (Lang et al., 2019) via a dedicated survival learner, providing users with additional access to standardized benchmarking, resampling, and tuning workflows.

Planned developments include expanded support for time-dependent evaluation metrics (Uno et al., 2007; Blanche et al., 2013) and the incorporation of additional survival loss functions. These directions aim to broaden the applicability of **survdnn** while preserving its core emphasis on transparent and statistically principled modeling.

Overall, **survdnn** provides a practical and extensible foundation for deep survival modeling in R, combining modern deep learning methodology with the transparency and reproducibility expected in statistical computing environments.

5 Acknowledgements

The author thanks the anonymous referees for their valuable comments and suggestions, which substantially improved the quality and clarity of this manuscript.

References

S. Batson, G. Greenall, and P. Hudson. Review of the reporting of survival analyses within randomised controlled trials and the implications for meta-analysis. *PLoS One*, 11(5): e0154870, 2016. [p1]

Y. Bengio. Practical recommendations for gradient-based training of deep architectures. In *Neural networks: Tricks of the trade: Second edition*, pages 437–478. Springer, 2012. [p7]

Y. Bengio, Y. Lecun, and G. Hinton. Deep learning for AI. *Communications of the ACM*, 64(7): 58–65, 2021. [p1, 15]

P. Blanche, J.-F. Dartigues, and H. Jacqmin-Gadda. Estimating and comparing time-dependent areas under receiver operating characteristic curves for censored event times with competing risks. *Statistics in Medicine*, 32(30):5381–5397, 2013. [p15]

N. E. Breslow. Contribution to discussion of paper by DR Cox. *Journal of the Royal Statistical Society, Series B*, 34:216–217, 1972. [p4]

D. R. Cox. Regression models and life-tables. *Journal of the Royal Statistical Society: Series B (Methodological)*, 34(2):187–202, 1972. [p1, 3, 15]

D. Falbel and J. Luraschi. *torch: Tensors and Neural Networks with 'GPU' Acceleration*, 2025. URL https://torch.mlverse.org/docs. R package version 0.15.0. [p2, 15]

D. Faraggi and R. Simon. A neural network model for survival data. *Statistics in Medicine*, 14(1):73–82, 1995. [p1]

J. H. Friedman, T. Hastie, and R. Tibshirani. Regularization paths for generalized linear models via coordinate descent. *Journal of Statistical Software*, 33(1):1–22, 2010. [p4, 7]

M. F. Gensheimer and B. Narasimhan. A scalable discrete-time survival model for neural networks. *PeerJ*, 7:e6257, 2019. [p7]

E. Graf, C. Schmoor, W. Sauerbrei, and M. Schumacher. Assessment and comparison of prognostic classification schemes for survival data. *Statistics in Medicine*, 18(17-18): 2529–2545, 1999. [p6]

F. E. Harrell, R. M. Califf, D. B. Pryor, K. L. Lee, and R. A. Rosati. Evaluating the yield of medical tests. *JAMA*, 247(18):2543–2546, 1982. [p6]

P. J. Heagerty and Y. Zheng. Survival model predictive accuracy and ROC curves. *Biometrics*, 61(1):92–105, 2005. [p1]

T. Hothorn, P. Bühlmann, S. Dudoit, A. Molinaro, and M. J. Van Der Laan. Survival ensembles. *Biostatistics*, 7(3):355–373, 2006. [p15]

H. Ishwaran, U. B. Kogalur, E. H. Blackstone, and M. S. Lauer. Random survival forests. 2008. [p15]

J. L. Katzman, U. Shaham, A. Cloninger, J. Bates, T. Jiang, and Y. Kluger. DeepSurv: personalized treatment recommender system using a Cox proportional hazards deep neural network. *BMC Medical Research Methodology*, 18(1):24, 2018. [p1, 3]

S. Keydana. *Deep learning and scientific computing with R torch*. Chapman and Hall/CRC, 2023. [p6]

H. Kvamme, Ø. Borgan, and I. Scheel. Time-to-event prediction with neural networks and Cox regression. *Journal of Machine Learning Research*, 20(129):1–30, 2019. [p1, 3, 4, 5]

M. Lang, M. Binder, J. Richter, P. Schratz, F. Pfisterer, S. Coors, Q. Au, G. Casalicchio, L. Kotthoff, and B. Bischl. mlr3: A modern object-oriented machine learning framework in R. *Journal of Open Source Software*, dec 2019. doi: 10.21105/joss.01903. URL https://joss.theoj.org/papers/10.21105/joss.01903. [p15]

C. Lee, W. Zame, J. Yoon, and M. Van Der Schaar. Deephit: A deep learning approach to survival analysis with competing risks. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 32, 2018. [p1]

M. Monod, P. Krusche, Q. Cao, B. Sahiner, N. Petrick, D. Ohlssen, and T. Coroller. Torchsurv: A lightweight package for deep survival analysis. *arXiv preprint arXiv:2404.10761*, 2024. [p1]

A. Mukherjee, R. Russell, S.-F. Chin, B. Liu, O. Rueda, H. Ali, G. Turashvili, B. Mahler-Araujo, I. Ellis, S. Aparicio, et al. Associations between genomic stratification of breast cancer and centrally reviewed tumour pathology in the METABRIC cohort. *npj Breast Cancer*, 4(1):5, 2018. [p14]

A. Paszke, S. Gross, F. Massa, A. Lerer, J. Bradbury, G. Chanan, T. Killeen, Z. Lin, N. Gimelshein, L. Antiga, et al. Pytorch: An imperative style, high-performance deep learning library. *Advances in Neural Information Processing Systems*, 32, 2019. [p1]

D. J. Sargent. Comparison of artificial neural networks with other statistical approaches: results from medical data sets. *Cancer: Interdisciplinary International Journal of the American Cancer Society*, 91(S8):1636–1642, 2001. [p1]

R. Sonabend. *survivalmodels: Models for Survival Analysis*, 2024. URL https://CRAN.R-project.org/package=survivalmodels. R package version 0.1.191. [p2]

Terry M. Therneau and Patricia M. Grambsch. *Modeling Survival Data: Extending the Cox Model*. Springer, New York, 2000. ISBN 0-387-98784-3. [p9]

H. Uno, T. Cai, L. Tian, and L.-J. Wei. Evaluating prediction rules for t-year survivors with censored regression models. *Journal of the American Statistical Association*, 102(478):527–537, 2007. [p15]

K. Ushey, J. Allaire, and Y. Tang. *reticulate: Interface to 'Python'*, 2025. URL https://github.com/rstudio/reticulate. R package version 1.42.0.9000, commit 3e1fa25e27d151a269172379e678b59c100221f9. [p2]

S. Varma and R. Simon. Bias in error estimation when using cross-validation for model selection. *BMC Bioinformatics*, 7(1):91, 2006. [p6]

H. Wickham. *ggplot2: Elegant Graphics for Data Analysis*. Springer-Verlag New York, 2016. ISBN 978-3-319-24277-4. URL https://ggplot2.tidyverse.org. [p3]

S. Wiegrebe, P. Kopper, R. Sonabend, B. Bischl, and A. Bender. Deep learning for survival analysis: a review. *Artificial Intelligence Review*, 57(3):65, 2024. [p1, 3]

A. Xiang, P. Lapuerta, A. Ryutov, J. Buckley, and S. Azen. Comparison of the performance of neural network methods and Cox regression for censored survival data. *Computational Statistics & Data Analysis*, 34(2):243–257, 2000. [p1]

H. Zhou, H. Wang, S. Wang, and Y. Zou. SurvMetrics: An R package for predictive evaluation metrics in survival analysis. *R Journal*, 14(4):252–263, 2023. [p6]

Imad El Badisy
AI and Data Science Service, Mohammed VI Center for Research and Innovation (CM6RI)
Rabat, Morocco
ORCiD: 0000-0003-4848-4895
elbadisyimad@gmail.com